Project 4-2
Implementation of a Code Generator II
Due on Friday, January 3, 2025

1. **Problem:**

Continue the project 4-1 to finish the code generation for if-then statements, if-then-else statements, and while statements in this project.

Consider the following **Cactus** program:

```
// A program to sum 1 to n
    Program
    Begin
        Var  n
        Var  s

        Read n
        If n < 0 Then
            Write -1
            Exit
        Else
            Set s = 0
        EndIf
        While n > 0 Do
            Set s = s + n
            Set n = n – 1
        EndWhile
        Write s
    End
```

The generated MIPS assembly code should be as follows:

```
        .data
n:      .word  0
s:      .word  0
        .text
main:                           # Begin
        li      $v0, 5          # Read n
```

```
        syscall
        la      $t0, n
        sw      $v0, 0($t0)
        la      $t0, n          # If n < 0
        lw      $t0, 0($t0)
        li      $t1, 0
        blt     $t0, $t1, L1
        b       L2
L1:     # Then
        li      $t0, 1          # Write -1
        neg     $t0, $t0
        move    $a0, $t0
        li      $v0, 1
        syscall
        li      $v0, 10         # Exit
        syscall
        b       L3
L2:     # Else
        li      $t0, 0          # Set s = 0
        la      $t1, s
        sw      $t0, 0($t1)
L3:     # EndIf
L4:     # While
        la      $t0, n          # n > 0
        lw      $t0, 0($t0)
        li      $t1, 0
        bgt     $t0, $t1, L5
        b       L6
L5:     # Do
        la      $t0, s          # Set s = s + n
        lw      $t0, 0($t0)
        la      $t1, n
        lw      $t1, 0($t1)
        add     $t0, $t0, $t1
        la      $t1, s
        sw      $t0, 0($t1)
        la      $t0, n          # Set n = n - 1
        lw      $t0, 0($t0)
```

```
        li      $t1, 1
        sub     $t0, $t0, $t1
        la      $t1, n
        sw      $t0, 0($t1)
        b       L4
L6:     # EndWhile
        la      $t0, s              # Write s
        lw      $t0, 0($t0)
        move    $a0, $t0
        li      $v0, 1
        syscall
        # End
```

2. Handing in your program

The code generator should be implemented by the C file codegen.c. The executable file should be named "cactus.exe" namely,

```
flex cactus.l
bison -d cactus.y
gcc -o cactus.exe lex.yy.c lotus.tab.c tree.c codegen.c
type input.txt | .\cactus.exe
```

To turn in the project, upload a flex file cactus.l, a bison file cactus.y, and a C file codegen.c to eCourse2 site.

3. Grading

The grading is based on the correctness of your program. The correctness will be tested by a number of test cases.

It is best to incrementally build your program so that you always have a partially-correct working program. It is also best to construct a shell script to systematically test your program.